
REPORT · 2026-09-28

MAPA COMPLETO DE GAPS — Alberto AI v1.7

71 gaps identificados em 6 categorias

AUTHOR

Mavis

DATE

2026-09-28



Resumo Executivo

Categoria	Total gaps	Críticos	Médios	Baixos
Segurança	8	3 (resolvidos em v1.7)	3	2
Funcionalidade	22	4	8	10
Cobertura de Testes	14	5	5	4
Operacional	12	4	5	3
UX/CLI	9	1	5	3
Documentação	6	0	3	3
TOTAL	71 gaps	17	29	25



CATEGORIA 1: SEGURANÇA (8 gaps)

✓ RESOLVIDOS em v1.7

- ✓ G-S1: `tool_file_write` sem NemoClaw → **FIXED**
- ✓ G-S2: `tool_run_shell` sem whitelist → **FIXED** (10+ blocked patterns)
- ✓ G-S3: "Comando:" interceptor executa shell puro → **FIXED** (whitelist validation)
- ✓ G-S4: Sem audit log → **FIXED** (`alberto/runtime/audit.py`)
- ✓ G-S5: Testes de regressão de segurança → **21 testes criados**
- ✓ G-S6: Documentação de segurança → **SECURITY-AUDIT.pdf**



AINDA EM ABERTO

G-S7: Lockfile em `/tmp/` não-isolado

- **Severidade:** 🟡 Média
- **Descrição:** `bin/alberto-serve` usa `/tmp/alberto-serve.pid` . Em sistema multi-user, attacker pode criar arquivo malicioso.
- **Workaround:** Hardcoded `$USER` no path seria mais seguro
- **Recomendação:** Usar `$XDG_RUNTIME_DIR/alberto/alberto-serve.pid`

G-S8: Falta de `--strict` mode para confirmação explícita

- **Severidade:** ● Baixa
 - **Descrição:** Não tem modo que requer user explicit "yes" para QUALQUER tool/shell
 - **Recomendação:** Adicionar flag `--strict` ao Alberto() que requer confirmação para qualquer write
-



CATEGORIA 2: FUNCIONALIDADE (22 gaps)

● CRÍTICOS

G-F1: Image generation (Flux 2 / SDXL) NÃO FUNCIONA

- **Severidade:** ● Crítica
- **Descrição:** Testei 7+ chaves NVIDIA diferentes, 18+ endpoints. Todos retornam 404 ou sem permissão.
- **Impacto:** Alberto NÃO consegue gerar imagens reais (apenas descreve)
- **Workaround parcial:** `black-forest-labs/flux.1-dev` funciona com a chave "Qwen" (não é realmente Qwen), mas outras chaves falham
- **Para resolver:** Conta NVIDIA AI Enterprise dedicada, ou usar API externa (Replicate/Stability)

G-F2: Hermes voice (TTS/STT) NÃO FUNCIONAL

- **Severidade:** ● Crítica
- **Descrição:** `tool_voice_tts`, `tool_voice_stt`, `tool_voice_mode` precisam de credenciais Magpie/ElevenLabs/OpenAI TTS que não temos
- **Impacto:** 3 tools retornam erro imediatamente
- **Para resolver:** API key Magpie + voz customizada

G-F3: Hermes xAI search NÃO FUNCIONAL

- **Severidade:** ● Crítica
- **Descrição:** `tool_x_search` precisa de xAI API key
- **Impacto:** Search avançado via xAI não disponível
- **Para resolver:** Adicionar key xAI

G-F4: NemoClaw full plugin NÃO INSTALÁVEL

- **Severidade:** ● Crítica
- **Descrição:** NemoClaw plugin oficial requer Docker + OpenClaw runtime que não temos
- **Impacto:** Apenas 6/36 NemoClaw features portadas (security_whitelist + sandbox detection)
- **Para resolver:** Docker + OpenClaw runtime disponível

MÉDIOS

G-F5: AIOX real CLI não existe

- 690 arquivos YAML mas ZERO código Python executável
- Documentado em `tests/test_v15_components.py` mas não testável

G-F6: 32 de 38 Hermes tools precisam de credenciais pagas

- discord, telegram, slack, whatsapp, feishu, signal, imessage, sms, wechat, email
- homeassistant (HomeAssistant instance)
- vision (HTTP 410 na API)
- image_gen (não configurado)
- 7 dos 38 tools funcionais sem credenciais: terminal, file, code_execution, memory, cron, browser (fallback urllib), patch_parser

G-F7: Sandbox-level isolation fraco

- `LocalSandbox` runs no host (não container)
- Sem protection de CPU/memory/disk
- Sandbox cleanup regex frágil (só deleta UUID-prefixed)

G-F8: MiMo primary engine binary não buildável

- `_try_build_binary` tenta buildar Node.js mas fonte está em `packages/opencode/src/`, não no path esperado
- Cai sempre pra FTS5-only fallback
- MiMo "max" mode (multi-model speculative) NÃO funciona

G-F9: AIOX workflow execution não testado

- 3 YAMLS existem mas execução real não foi validada
- `run_aiox_workflow()` em `integration.py` provavelmente é stub

G-F10: FTS5 memory limitada

- Apenas basic CRUD (set/get/list)
- Sem search semântico
- Sem TTL/expiração
- Sem quota management

G-F11: Self-healing fixers só logam

- 15 fixers implementados mas só escrevem no `heal_log.json`
- AUTO-APLICAÇÃO raramente acontece
- Sem modo `--apply-heals` que re-executa código após fix

G-F12: Squad system só discovery

- Carrega 23 personas + 12 workflows
- Mas `run_squad()` é stub que só executa 1 persona
- Multi-persona pipeline quebrado

BAIXOS

G-F13-G22 (10 gaps menores):

- `agent.run_squad()` não executa parallel personas
- `meta_controller.plan()` retorna placeholder
- `learner.create_skill()` só salva markdown (não executa training)
- `researcher.search()` sem provider key
- `mcp.server()` subprocess management tem `except: pass` silencioso
- `extensions.install()` procura mas não instala
- `autonomy.loop()` queue management simplificado
- `tools_registry` é só 15 linhas (pass-through)
- `server.app` 30+ endpoints, 0 testes
- `cli.py` 901 linhas, 7% coverage

CATEGORIA 3: COBERTURA DE TESTES (14 gaps)

Módulos CRÍTICOS com cobertura < 20%

Módulo	LoC	Coverage	Gap
<code>cli.py</code>	901	7%	G-T1: 40+ subcommands SEM teste
<code>engines/hermes_tools.py</code>	1774	12%	G-T2: 38 tools, 0 testes integrados
<code>agent.py</code>	612	17%	G-T3: pipeline agent() não testado
<code>runtime/integration.py</code>	172	12%	G-T4: Hermes+AIOX+NemoClaw bridge não testado
<code>runtime/function_caller.py</code>	646	14%	G-T5: Tool registry sem contract test

Módulos com 0 testes diretos

- **G-T6:** `autonomy.py` (24/7 loop) — não testado

- **G-T7:** `learner.py` — não testado
- **G-T8:** `meta_controller.py` — não testado
- **G-T9:** `researcher.py` — não testado
- **G-T10:** `app_creator.py` — apenas smoke test

● TESTES FALTANDO por area

- **G-T11:** Property-based tests para Hermes tools (hypothesis para inputs arbitrários)
 - **G-T12:** Contract tests para OpenAI API (mocks vs real shape)
 - **G-T13:** Mutation tests com mutmut (para validar quality dos asserts)
 - **G-T14:** Integration tests para alberto-serve em modo real (subprocess + curl)
-

| 🚀 CATEGORIA 4: OPERACIONAL (12 gaps)

● CRÍTICOS

G-O1: Sem CI/CD pipeline (GitHub Actions)

- Nenhum workflow em `.github/workflows/`
- Tests não rodam automaticamente em PR
- Sem quality gates

G-O2: Sem Dockerfile oficial

- Alberto roda in-host (não container)
- NemoClaw plugin requer Docker mas não temos config

G-O3: Sem lockfile de versões Python (poetry.lock / uv.lock)

- `requirements.txt` tem apenas `>=` (sem upper bounds)
- Conflitos de dependência possíveis

G-O4: Sem monitoramento em produção

- Sem health checks (exceto `alberto-serve /api/status`)
- Sem métricas (Prometheus/etc)
- Sem alertas

● MÉDIOS

G-O5: Sem release pipeline

- Releases são manuais (`gh release create`)

- Sem versionamento semântico automatizado

G-O6: Sem changelog

- Sem `CHANGELOG.md` ou `RELEASES.md`

G-O7: Sem backup/restore procedure

- `~/.alberto/audit.log` e `conversations.json` sem backup
- Sem git LFS para skills

G-O8: Sem rate limiting

- Sem proteção contra abuse no fastapi server
- LLM pode ser chamado unlimited

G-O9: Sem multi-tenancy

- Lockfile em `/tmp/` não isola users
- Sem per-user config

BAIXOS

G-O10: Sem license header check

- Files sem SPDX license identifier

G-O11: Sem CONTRIBUTING.md / CODE_OF_CONDUCT.md

- Sem guide para contribuidores

G-O12: Pre-flight install só Aliyun/Tsinghua/PyPI

- Sem mirrors próprios (corp intranet)

CATEGORIA 5: UX/CLI (9 gaps)

CRÍTICOS

G-UX1: CLI 0% testada

- 40+ subcommands, 7% coverage
- Sem `argcomplete` /autocomplete
- Sem `--json` output em vários comandos

● MÉDIOS

G-UX2: Help inconsistente

- Alguns comandos `--help` rico, outros pobres
- Sem exemplos de uso
- Falta grouping por categoria

G-UX3: Mensagens de erro confusas

- Erros críticos como `rc=2` sem explicação
- Sem error codes padronizados

G-UX4: Sem `--verbose` / `--debug`

- Debug é all-or-nothing (env var `ALBERTO_DEBUG=1`)
- Sem níveis progressivos (info, debug, trace)

G-UX5: Output JSON com chaves inconsistentes

- Alguns retornam `{"ok": true, ...}` , outros `{"result": ...}`
- Sem JSON Schema documentada

● BAIXOS

G-UX6: Sem tema/cores

- Output sempre plain text
- Sem highlight de keywords

G-UX7: Sem progress bars

- Operações longas (preflight_install, watchdog backoff) sem feedback visual

G-UX8: Sem `--dry-run`

- Não tem como testar comandos sem executar

G-UX9: Sem `--config FILE`

- Config hardcoded em `~/.alberto/config.json`
 - Sem override por env ou CLI flag
-



CATEGORIA 6: DOCUMENTAÇÃO (6 gaps)

● MÉDIOS

G-D1: Sem OpenAPI spec para FastAPI server

- 30+ endpoints em `server/app.py` mas sem `openapi.json` validado
- Sem Swagger UI

G-D2: Documentação dispersa em 12+ MD files

- `V1.1` a `V1.5b`, `FUNCTION-CALLING-RESULT.md`, `T2-T3-RESULTS.md`, etc.
- Sem índice central
- Sem "Quick Start" 5-min

G-D3: README muito curto

- 4KB mas sistema tem 11K LoC
- Falta architecture diagram
- Falta exemplos reais de uso

● BAIXOS

G-D4: Sem CONTRIBUTING.md

- Sem como contribuir

G-D5: Sem TROUBLESHOOTING.md

- Sem guia para erros comuns

G-D6: Sem MIGRATION.md

- Sem guia de upgrade entre versões
-



TOP 10 GAPS PRIORITÁRIOS (Próximos Sprints)

#	Gap	Categoria	Esforço	Impacto
1	G-T1: CLI tests (40+ subcommands)	Testes	3 dias	Quality
2	G-T2: Hermes tools integration tests	Testes	2 dias	Reliability
3	G-O1: GitHub Actions CI	Op	1 dia	Quality gates
4	G-F1: Image generation	Funcional	1-2 semanas	Feature
5	G-UX1: CLI usability	UX	2 dias	Developer experience
6	G-S7: Lockfile isolation	Security	0.5 dia	Security hardening
7	G-D1: OpenAPI spec	Docs	1 dia	API docs
8	G-O3: poetry.lock/uv.lock	Op	0.5 dia	Reproducible builds
9	G-F12: Squad pipeline (parallel)	Funcional	1 semana	Multi-agent
10	G-D3: README expansion	Docs	0.5 dia	Onboarding



STATUS POR MÓDULO

✅ Funcionando bem (15 módulos)

- `identity.py` (88% test)
- `security_whitelist.py` (novo, 100% funcionou)
- `audit.py` (novo, 100% funcionou)
- `personas/catalog.py` (data loading)
- `shortcuts/store.py` (CRUD simples)
- `sandbox/base.py` (LocalSandbox)
- `model_router.py` (OpenAI-compatible)
- `tools_registry.py` (pass-through)
- `engines/base.py` (interface)
- `engines/mimo.py` (FTS5 memory)
- `nemoclaw_real.py` (36 secret patterns)
- `smart_router_v2.py` (intent detection)
- `smart_router.py` (multi-model)

- `skill_engine.py` (779 skills)
- `conversation.py` (history)

⚠ Funcional mas parcial (8 módulos)

- `agent.py` (chat funciona mas tool loop parcial)
- `cli.py` (40+ subcommands, 7% test)
- `hermes_tools.py` (38 tools, 6 funcional, 32 stub)
- `function_caller.py` (32 schemas, dispatch OK)
- `integration.py` (exports OK, fallbacks OK)
- `engine/hermes.py` (engine wrapper)
- `runtime/smart_router.py` (basic routing)
- `runtime/extensions.py` (extension loader)

✗ Stubs/degradam (10 módulos)

- `runtime/integration.py` (Mavis fallback com warnings)
 - `runtime/app_creator.py` (basic scaffold)
 - `runtime/autonomy.py` (loop não testado)
 - `runtime/learner.py` (skill gen sem validation)
 - `runtime/mcp.py` (subprocess `except: pass`)
 - `runtime/meta_controller.py` (plan retorna placeholder)
 - `runtime/researcher.py` (sem provider)
 - `runtime/self_healing.py` (15 fixers, auto-apply raro)
 - `runtime/tester.py` (smoke test apenas)
 - `server/app.py` (30+ endpoints, 25% test)
-



Métricas

Métrica	Valor
Total LoC Python	11,412
Total arquivos Python	39
Módulos runtime	15
Hermes tools definidos	38 (6 funcionais sem creds)
Tools no function_caller	32 (todas funcionais)
Skills carregadas	779 (10 auto-invoke patterns)
Personas	23 .md files
Workflows	11 YAMLS
Tests passing	229
Test coverage	20% geral
Critical coverage	52-100% em módulos críticos
Secrets detectados	36 patterns
Blocked shell patterns	10 patterns



Recomendação Final

v1.8 (curto prazo, 1 sprint)

1. CLI 100% testada (G-T1)
2. GitHub Actions (G-O1)
3. OpenAPI spec (G-D1)
4. poetry.lock (G-O3)
5. Lockfile isolation (G-S7)
6. README expansion (G-D3)

v2.0 (médio prazo, 2-3 sprints)

1. Image generation real (G-F1) — requer NVIDIA Enterprise account

2. Hermes voice/image tools (G-F2) — requer credenciais pagas
3. NemoClaw full plugin (G-F4) — requer Docker + OpenClaw
4. Squad parallel execution (G-F12)
5. CLI 100% cobertura + OpenAPI + Swagger UI

v3.0 (longo prazo)

1. Multi-tenancy com Docker
2. Cloud-native (k8s)
3. SaaS offering
4. Pro features (image gen, voice, web automation)

Total identificado: 71 gaps em 6 categorias

Já resolvidos: 6 gaps de segurança (v1.7) **Para v1.8:** 17 gaps críticos/médios prioritários